

Chapter 9

Simulation Software

Overview

9.1 Simulation in Java

9.2 Simulation in GPSS

9.3 Simulation in Python

9.4 Other Simulation Software

History of Simulation Software

- Software used to develop simulation models can be divided into 3 categories.
 - First there are the general purpose programming language such as C, C++ and Java.
 - Second there are simulation programming language such as GPSS/H, SIMAN V and SLAM II.
 - Third there are the simulation environment. This category includes many products that are distinguished in one way or other by cost, application area or type of animation but have common characteristics such as a graphical user interface.

History of Simulation Software

- Our discussion in the history of simulation software is based on Nance[1995], who breaks the years from 1955 through 1986 into five periods.
- 1955 - 60 The Period of Search
 - Researchers searched for common simulation concepts and developed reusable routines.
 - Most simulation programs were written in FORTRAN without specialized simulation libraries.
- 1961 - 65 The Advent
 - Appearance of the forerunners of simulation programming languages (SPLs.)
 - The first process interaction SPL, GPSS was developed at IBM for quick simulation of communication and computer systems.

History of Simulation Software

- 1966 - 70 The Formative Period
 - Simulation concepts were reviewed and improved to make languages more consistent.
 - Rapid hardware advancements and user demands forced some languages to undergo major revisions.
- 1971 - 78 The Expansion Period
 - Major advances in GPSS came from outside IBM
 - GPSS/NORDEN, a pioneering effort that offered an interactive, visual online environment (in Norden Systems.)
 - GASP added support for the activity-scanning worldview and event scheduling worldview (at Purdue.)

History of Simulation Software

- 1979 - 86 The Period of Consolidation and Regeneration
 - Beginnings of SPLs written for, or adapted to, desktop computers and microcomputers.
 - Two major descendants of GASP appeared: SLAM II and SIMAN (provide multiple modeling perspectives and combined modeling capabilities).
- 1987 – Now The Period of Integrated Environments
 - Growth of SPLs on the personal computer and the emergence of simulation environments with graphical user interfaces, animation and other visualization tools.
 - Many of these environments also contain the input-data analyzers and output analyzers.
 - Animation ranges from 2-D and 3-D scale drawings.
 - Recent advancements have been made in web-based simulation.

Selection of Simulation Software

- Advice when evaluating and selecting simulation software:
 - Do not focus on single issue, such as ease of use. Consider the accuracy and level of detail obtainable, ease of learning, vendor support, and applicability to your applications.
 - Execution speed is important.
 - Beware of advertising claims and demonstrations.
 - Ask the vendor to solve a small version of your problem.

Selection of Simulation Software

- Beware of “checklists” with “yes” and “no” as the entries, e.g. many packages claim to have a conveyor entity, however, implementations have considerable variation and level of fidelity.
- Determine whether the simulation package and language are sufficiently powerful to avoid having to write logic in any external language.
- Beware of “no programming required,” unless either the package is a near-perfect fit to your problem domain, or programming is possible with the supplied blocks, nodes, or process-flow diagram.

9.1 Simulation in Java

- Java is widely used programming language that has been used extensively in simulation.
- It does not provide any facilities directly aimed at aiding the simulation analyst.
- The runtime library provides a random-number generator.
- It supports modular construction of large models.
- Simulation libraries such as SSG alleviate the development burden.
- Provides access to standardized simulation functions and hide low-level scheduling minutiae.

Simulation in Java

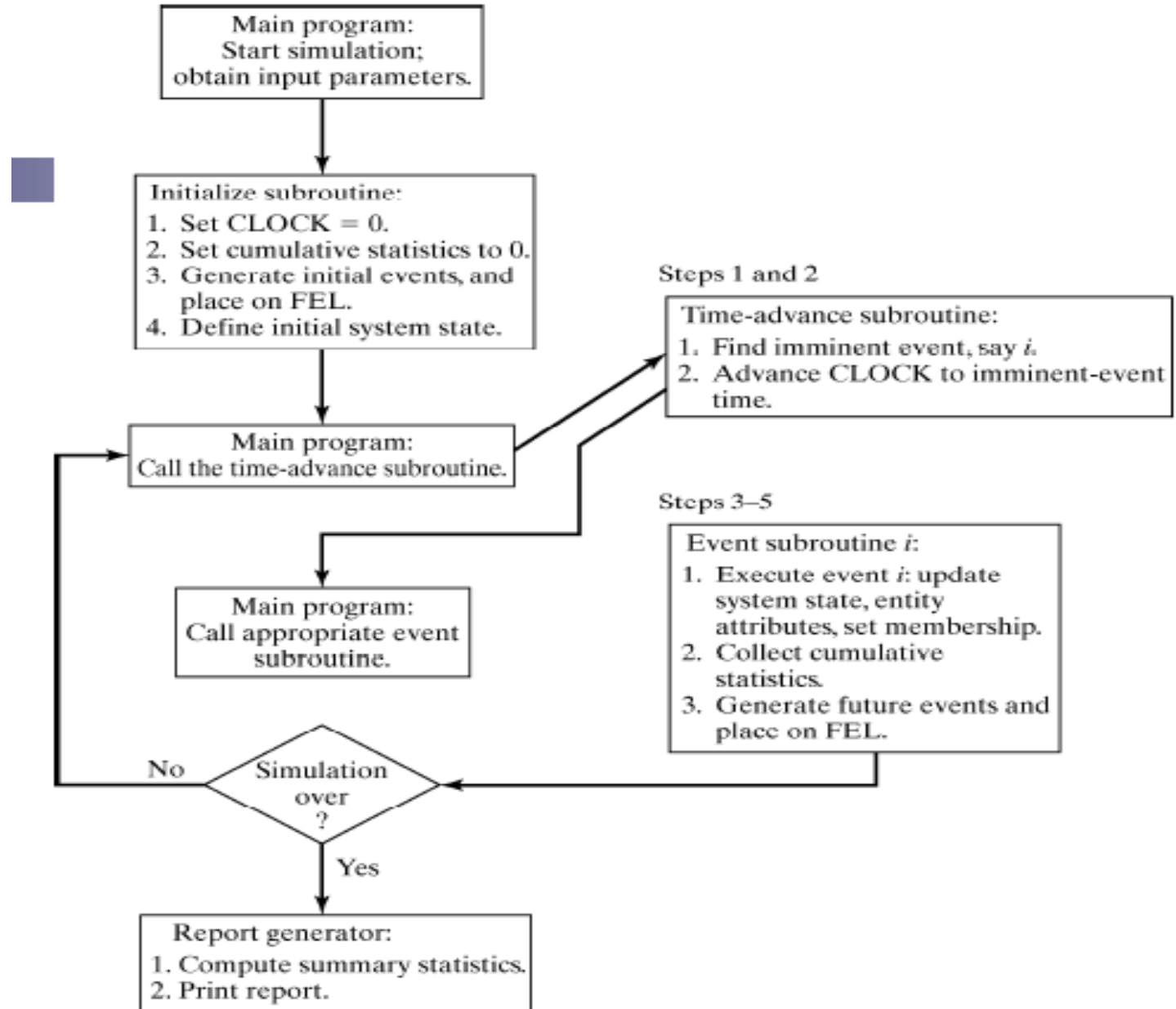
- Discrete-event simulation model written in Java contains the following :
 - Basic components:
 - System state
 - Entities and attributes
 - Sets
 - Events
 - Activities
 - Delays

Simulation in Java

- Common components (when organizing model in a modular fashion by using methods):
 - Clock- a variable defining simulated time.
 - Initialization method – a method to define the system state at time 0.
 - Min-time event method - a method that defines the element of future event list that has the smallest time-stamp.
 - Event methods – A separate method for each type of event, updates the system state when that event occurs.
 - Random-variate generators – method to generate samples from desired probability distributions.
 - Main program – to maintain the overall control of the event-scheduling algorithm.
 - Report generator – a method that computes summary statistics from cumulative statistics and prints a report at the end of the simulation.

Simulation in Java

- The overall structure of Java simulation is:

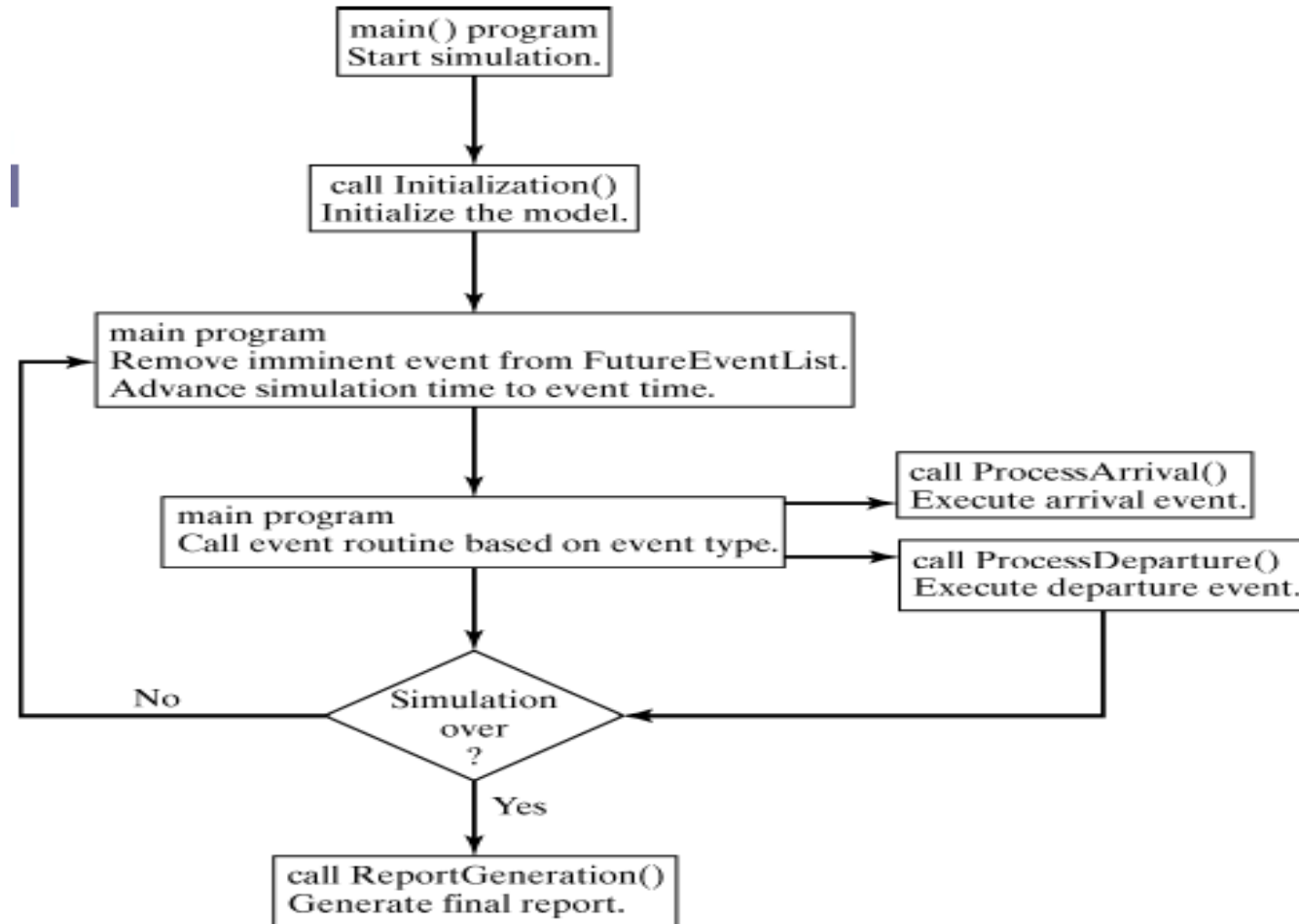


Simulation in Java

Ex: A Grocery Checkout Counter: A Typical Single Server Queue

The system a grocery checkout counter, is modelled as a single server queue. The simulation will run until 1000 customers have been served. In addition, assume that the inter-arrival time of the customers is exponentially distributed with a mean of 4.5 minutes, and that the service time are normally distributed with a mean of 3.2 minutes and a standard deviation of 0.6 minute. When the cashier is busy, a queue forms with no customers turned away.

Simulation in Java: Single-Server Queue Example



The Checkout Counter: Variables, Functions and Subroutines Involved

Variables	Descriptions
System State: QueueLength NumberInServices	 Number of customers in queue at current simulated time Number of customer being served at current simulated time
Entity Attributes and Sets: Customers	FCFS Queue of customer in system
Future Event List: FutureEventList	Priority-ordered list of pending event
Activity Durations: MeanInterArrivalTime MeanServiceTime	The inter-arrival time between previous customer's arrival and the next arrival The service time of the most recent customer to begin service

The Checkout Counter: Variables, Functions and Subroutines Involved

Variables	Description
Input Parameters: MeanInterArrivalTime MeanServiceTime SIGMA TotalCustomers	Mean inter arrival time, 4.5 minutes Mean service time, 3.2 minutes Standard deviation of service time, 0.6 minutes The stopping criteria- number of customers to be served, 1000
Simulation Variables: Clock Statistical Accumulators: LastEventTime TotalBusy MaxQueueLength SumResponseTime NumberOfDepatures LongService	The current value of simulated time Time of occurrence of last event Total busy time of server (so far) Maximum length of waiting line (so far) Sum of customer response times for all customers who have departed (so far) Number of departures (so far) Number of customers who spent 4 or more minutes at the checkout counter (so far)

The Checkout Counter: Variables, Functions and Subroutines Involved

Variables	Description
Summary Statistics: RHO = BusyTime/Clock AVGR PC4	Proportion of time server is busy Average Response Time Proportion of customers who spent 4 or more minutes at the checkout counter
Functions	Description
Exponential (mu) Normal (xmu, SIGMA)	Function to generate samples from an exponential distribution with mean mu Function to generate samples from a normal distribution with mean xmu and standard deviation SIGMA

The Checkout Counter: Variables, Functions and Subroutines Involved

Method	Description
Initialization	Initialization method
ProcessArrival	Event method that executes the arrival event
ProcessDepature	Event method that executes the departure event
ReportGeneration	Report Generator

Simulation in GPSS

- GPSS (General Purpose Simulation System) is a highly structured, special-purpose simulation programming language.
 - Based on the process-interaction approach.
 - Oriented toward queueing systems.
- Use of block diagram:
 - Provides a convenient way to describe the system.
 - With over 40 standard blocks.
 - Blocks represents events, delays and other actions that affect transaction flow.
- Block diagram is converted to block statements, control statements are added, and result in a GPSS model is obtained.

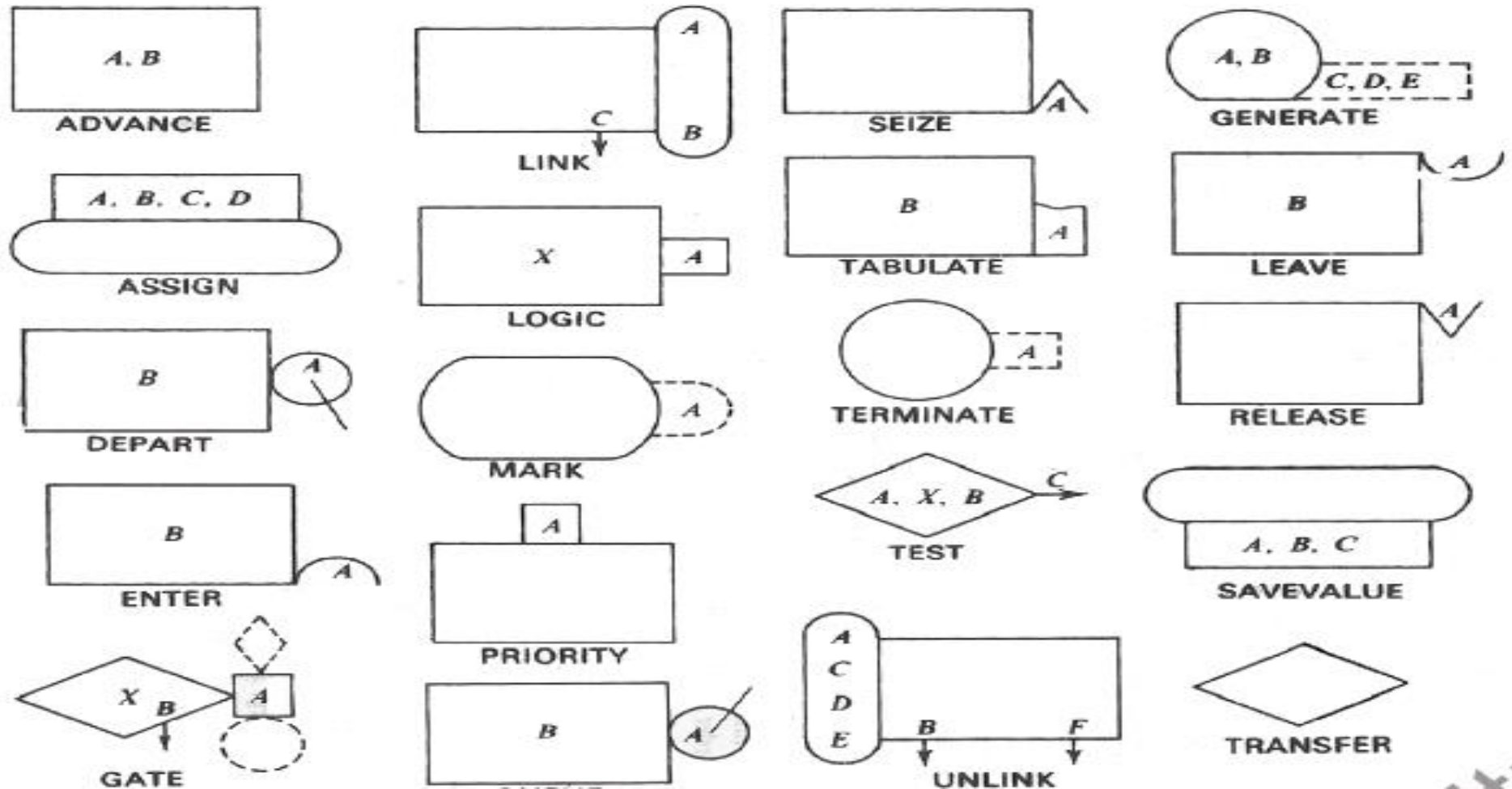
Simulation in GPSS

- The 1st version was released by IBM in 1961.
- GPSS/H is the most widely used version today.
 - Released in 1977
 - Flexible yet powerful.

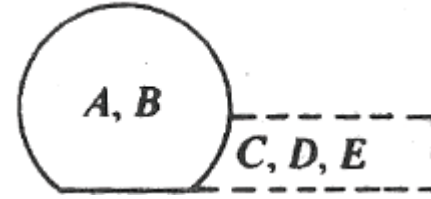
Simulation in GPSS: GPSS – Basic commands

- CLEAR - reset statistics and remove transaction
- CONTINUE - resume the simulation
- EXIT - end the GPSS world session
- HALT - stop the simulation and delete all queued commands
- INCLUDE - read and translate a secondary model file
- INTEGRATE - automatically integrate a time differential in a use variable
- REPORT - set the name of the report file or request an immediate report
- RESET - reset the statistics of the simulation
- SHOW - evaluate and display expression
- START - set the termination count and begin a simulation
- STEP - attempt a limited number of block entities
- STOP - set a stop condition based on block entry attempts
- STORAGE - define a storage entity
- TABLE - define a table entity
- VARIABLE - define a variable entity

Simulation in GPSS: GPSS – Blocks



GPSS – Blocks



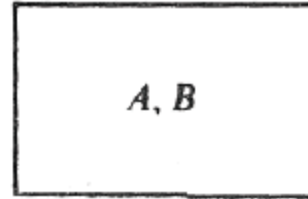
GENERATE

- A GENERATE Block creates Transactions for future entry into the simulation.

GENERATE A,B,C,D,E

- Example : GENERATE 0.1
- This is the simplest way to use the GENERATE Block. This Block causes a priority zero Transaction to enter the simulation every tenth of a time unit.

GPSS – Blocks



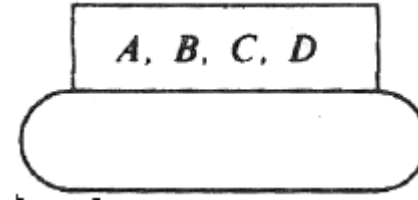
ADVANCE

- An ADVANCE Block delays the progress of a Transaction for a specified amount of simulated time.

ADVANCE A,B

- Example: ADVANCE 101.6, 50.3
- This example creates a Block which chooses a random number between 50.3 and 151.9, inclusively (i.e. 101.6 plus or minus 50.3), and delays the entering Transaction that amount of simulated time

GPSS – Blocks



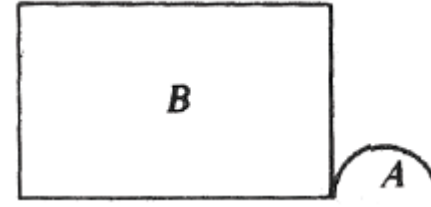
ASSIGN

- ASSIGN Blocks are used to place or modify a value in a Transaction Parameter.

ASSIGN A,B,C;

- Examples: ASSIGN 2000, 150.6
- This is the simplest way to use the ASSIGN Block. The value 150.6 is assigned to Parameter number 2000 of the entering Transaction. If no such Parameter exists, it is created.

GPSS – Blocks



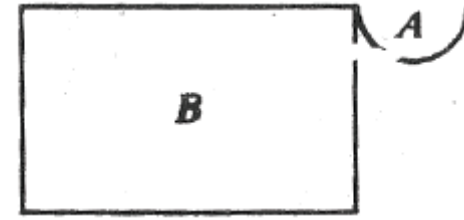
ENTER

- When a Transaction attempts to enter an ENTER Block, it either takes or waits for a specified number of storage units.

ENTER A,B

- Example: ENTER Toolkit, 2
- In this example the Active Transaction demands 2 storage units from the storage units available at the Storage Entity named Toolkit. If there are not enough storage units remaining in the Storage Entity, the Transaction comes to rest on the Delay Chain of the Storage Entity.

GPSS – Blocks



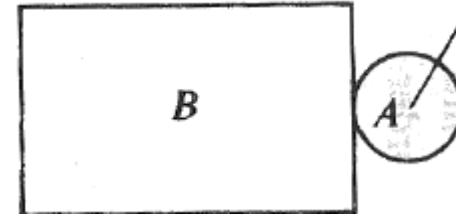
LEAVE

- A LEAVE Block increases the accessible storage units at a Storage Entity.

LEAVE A,B

- Example : LEAVE RepairMen,10
- In this example, when a Transaction enters the LEAVE Block, the available storage units at the Storage Entity named RepairMen is increased by 10.

GPSS – Blocks



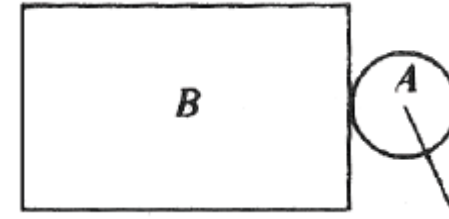
QUEUE

- A QUEUE Block updates Queue Entity statistics to reflect an increase in content.

QUEUE A,B

- Example : QUEUE WaitingLine
- In this example the content of the Queue Entity named WaitingLine is increased by one and the associated statistics accumulators are updated

GPSS – Blocks



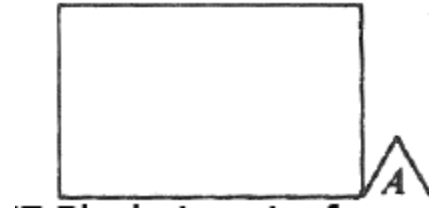
DEPART

- A DEPART Block registers statistics which indicate a reduction in the content of a Queue Entity.

DEPART A,B

- Example : DEPART WaitingLine
- In this example the content of the Queue Entity named WaitingLine is reduced by one and the associated statistics accumulators are updated.

GPSS – Blocks



SEIZE

- When the Active Transaction attempts to enter a SEIZE Block, it waits for or acquires ownership of a Facility Entity.

SEIZE A

- Example : SEIZE Teller1
- In this example, when a Transaction attempts to enter the SEIZE Block, the state of the Facility named Teller1 is tested. If it is idle, ownership is given to the Active Transaction, which is allowed to enter the SEIZE Block and proceed to the Next Sequential Block (NSB). If the Facility is busy (owned), the Active Transaction comes to rest on the Delay Chain of the Facility.

GPSS – Blocks



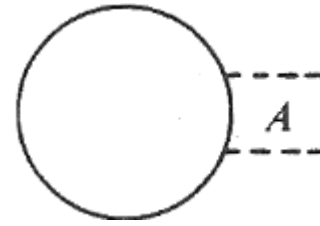
RELEASE

- A RELEASE Block releases ownership of a Facility, or removes a preempted Transaction from contention for a Facility.

RELEASE A

- Example : RELEASE Teller1
- In this example, when a Transaction which owns the Facility Entity named Teller1 enters the RELEASE Block, it gives up ownership to the Facility.

GPSS – Blocks



TERMINATE

- A TERMINATE Block removes the Active Transaction from the simulation and optionally reduces the Termination Count.

TERMINATE A

- Example : TERMINATE 1
- In this example, when a Transaction enters the TERMINATE Block it is removed from the simulation. Also, the Termination Count of the simulation, which is set by a START Command is decremented by 1.

GPSS – Blocks



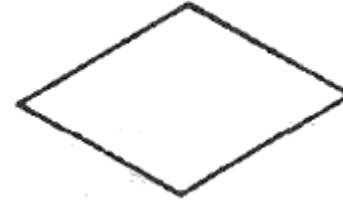
TEST

- A TEST Block compares values, and controls the destination of the Active Transaction based on the result of the comparison.

TEST O A,B,C

- Example : TEST G C1, 70000
- In this example of a "Refuse Mode" TEST Block, the Active Transaction enters the TEST Block if the relative system clock value is greater than 70000. Otherwise, the Transaction is blocked until the test is true.

GPSS – Blocks



TRANSFER

- A TRANSFER Block causes the Active Transaction to jump to a new Block location.

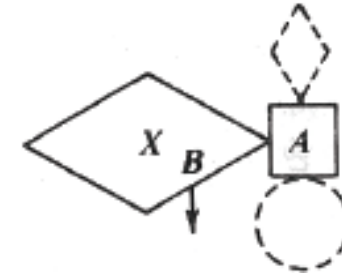
TRANSFER A,B,C

A: The probability of transactions directed to operand C

B: The label/destination for the remaining transactions

C: The label/destination for the transactions selected by probability A

GPSS – Blocks



GATE

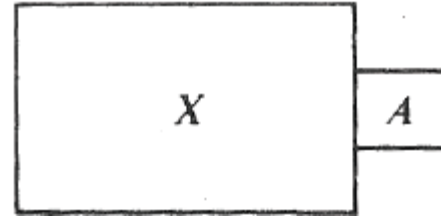
- A GATE Block alters Transaction flow based on the state of an entity.

GATE O A,B

Where, O is Operator mode(U, NU, SF, SNF, SE, SNE)

- Examples : GATE SNF MotorPool
- In this example of a "Refuse Mode" GATE Block, the Active Transaction enters the GATE Block if the Storage Entity named MotorPool is not full (i. e. if at least 1 unit of storage is available). If the Storage is full, the Active Transaction is blocked until 1 or more storage units become available.

GPSS – Blocks



LOGIC

- A LOGIC Block changes the state of a Logicswitch entity.

LOGIC O A

Where O= Operation (S, R, or I)

- Example : LOGIC S PowerSwitch
- In this example, the Logicswitch Entity named PowerSwitch is left in the true or "set" state.

GPSS – Blocks



MARK

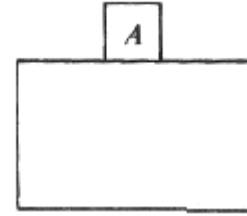
- A MARK Block records the current simulation time for a transaction into its Parameter.

MARK A

Where, A = Transaction parameter number in which the current simulation clock is stored.

- Examples : MARK Beginning
- In this example, when a Transaction enters the MARK Block, its Transaction Parameter named Beginning is given a value equal to the value of the absolute system clock.

GPSS – Blocks



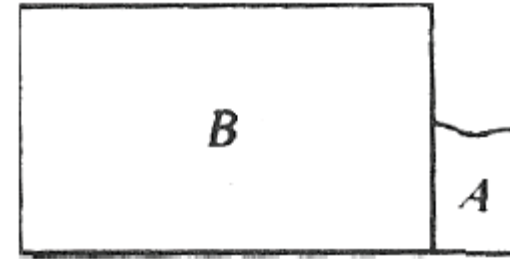
PRIORITY

- A PRIORITY Block sets the priority of the Active Transaction.

PRIORITY A

- Example : PRIORITY 10
- In this example any entering Transaction is made to have a priority of 10.

GPSS – Blocks



TABULATE

- A TABULATE Block triggers the collection of a data item in a Table Entity.

TABULATE A, B

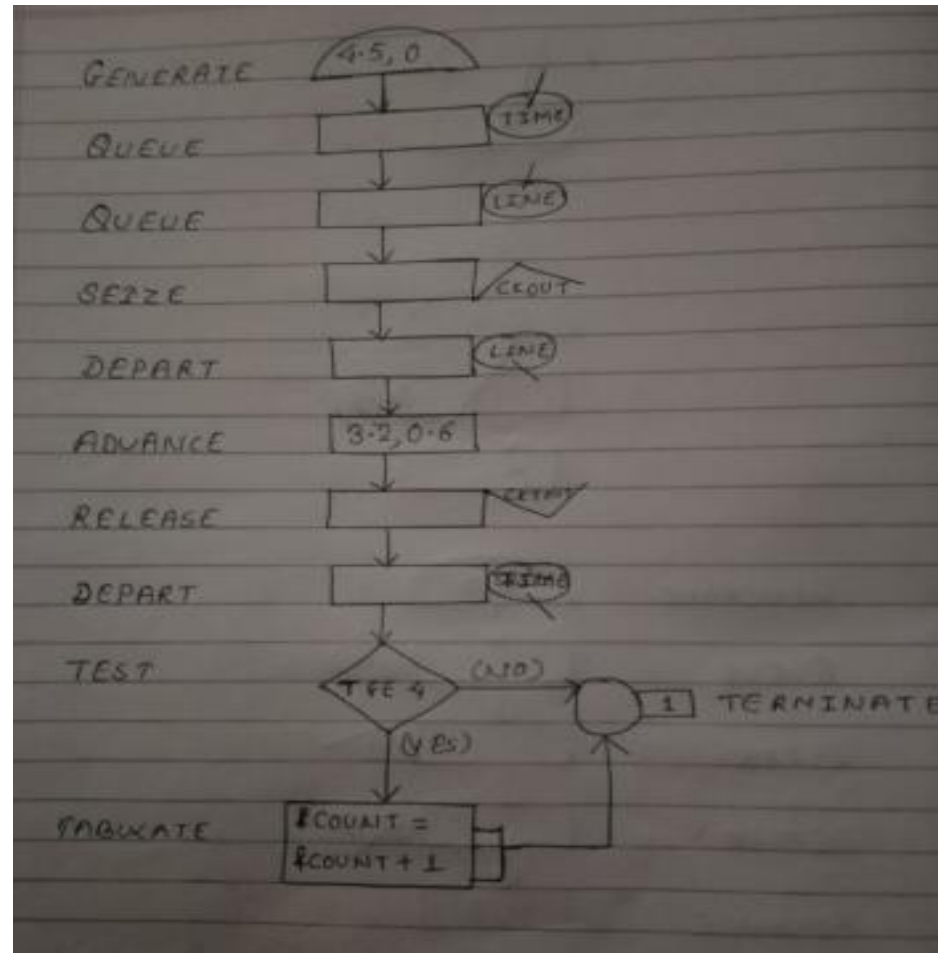
- Example : TABULATE Sales
- When a Transaction enters this TABULATE Block, the Table Entity named Sales is found. Sales must have been defined in a TABLE Command.

Example 1: Single Server Queue

Scenario: A Grocery Checkout Counter: A Typical Single Server Queue

The system a grocery checkout counter, is modelled as a single server queue. The simulation will run until 1000 customers have been served. In addition, assume that the inter-arrival time of the customers is exponentially distributed with a mean of 4.5 minutes, and that the service time are normally distributed with a mean of 3.2 minutes and a standard deviation of 0.6 minute. When the cashier is busy, a queue forms with no customers turned away.

Example 1: Single Server Queue

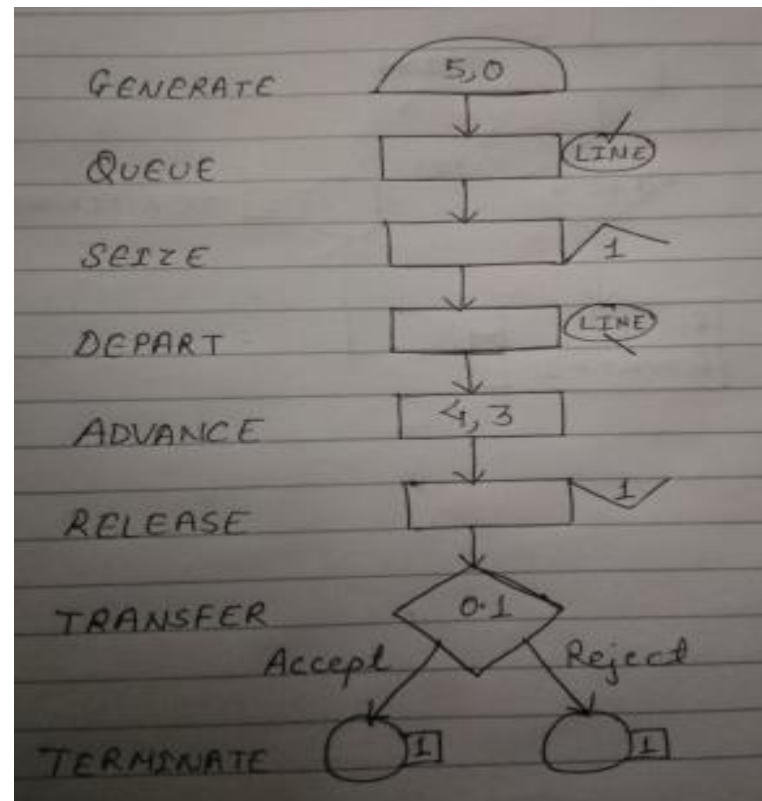


Example 2: Manufacturing Shop

- Scenario 1 : A machine tool in a manufacturing shop is turning out parts at the rate of one in every 5 minutes. As they are finished the parts go to an inspector and in the inspector takes 4 ± 3 mins to examine each one and rejects about 10% of parts. When the instructor is busy, a queue forms.

Example 2: Manufacturing Shop

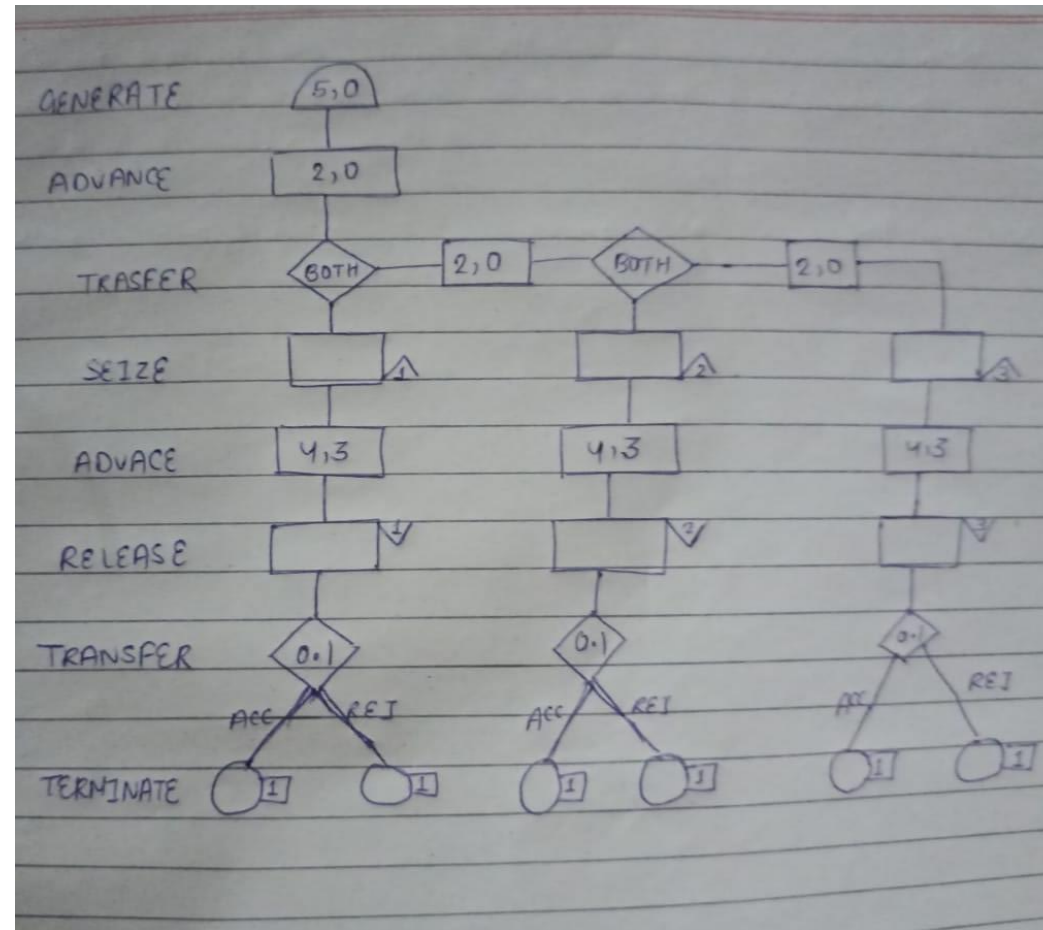
- Scenario 1 : A machine tool in a manufacturing shop is turning out parts at the rate of one in every 5 minutes. As they are finished the parts go to an inspector and in the inspector takes 4 ± 3 mins to examine each one and rejects about 10% of parts.



Example 2: Manufacturing Shop

- Scenario 2: consider the same example and also 3 inspectors for inspecting tasks. The manufactured parts are put on a conveyer which carries the parts to inspector placed at intervals of 2 minutes. It takes 2 minutes to reach the 1st inspector and if he is busy it takes additional 2 minutes to reach to the another inspector.

Example 2: Manufacturing Shop



Example 3: Supermarket

Scenario: Customers arrive at the supermarket, get a basket if available. If no basket is available he/she leaves without shopping. If the basket is available he/she shops and then checks out at the counter, returns basket and leaves the supermarket. Assume the rate of customers arrivals at Poisson's distribution. Also assume shopping time is linear with number of items to be served. The duration to pay bill is assumed as $y = -\ln(1-x)$.

➤ Solution,

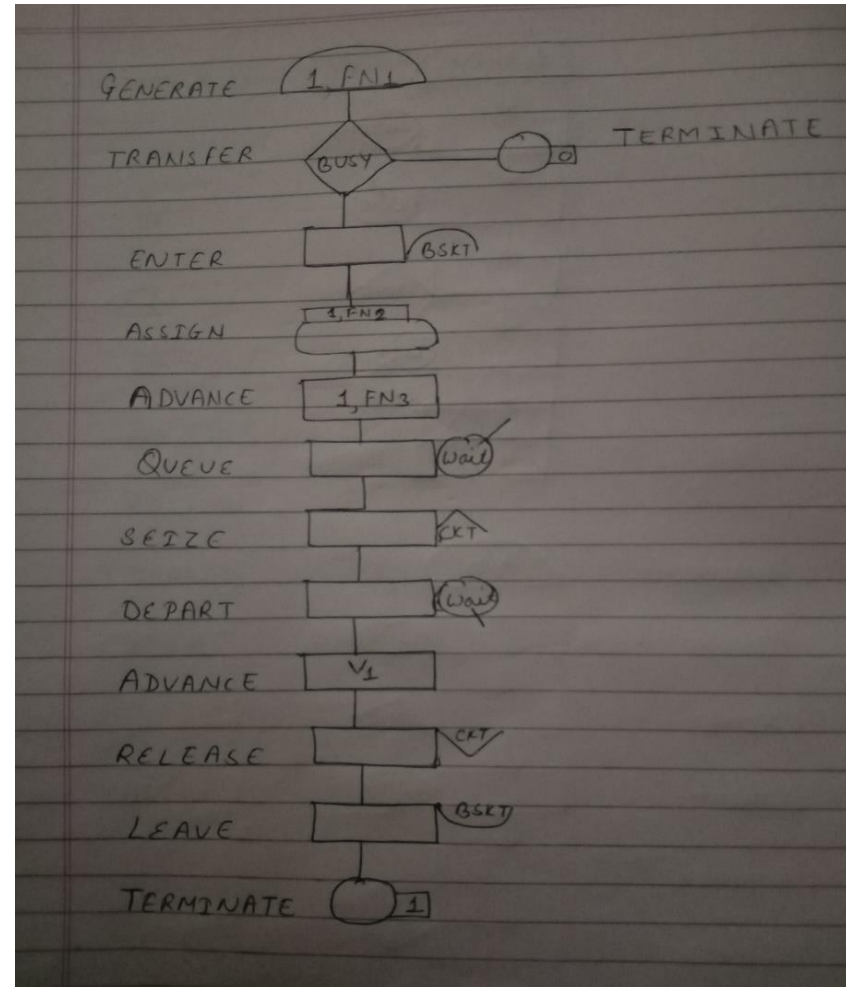
Let FN1 = Poisson distribution

FN2 = function used to obtain number of items for shoppers to shop

FN3 = linear function

V1 = time to pay bill [$y = -\ln(1-x)$]

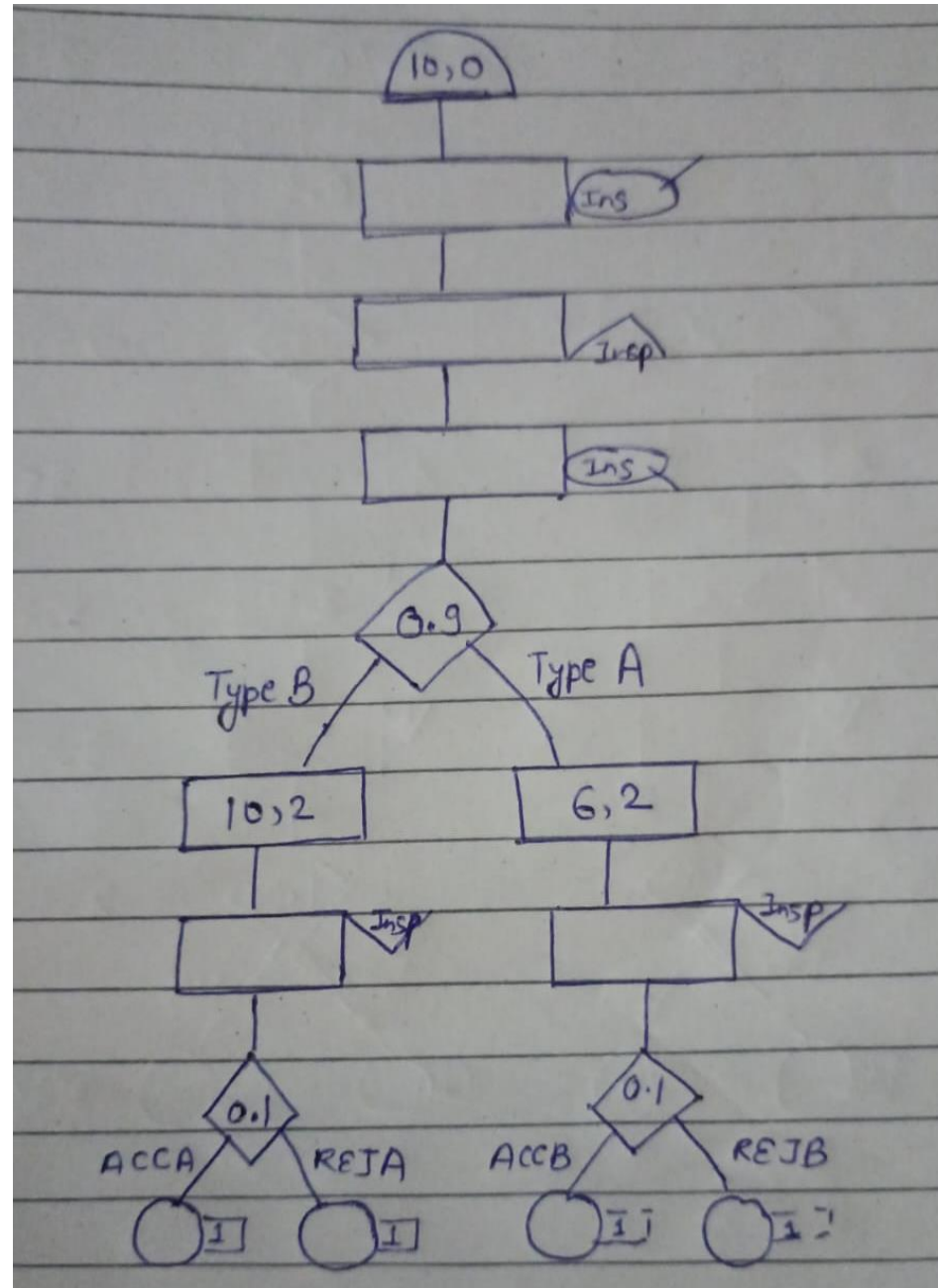
Example 3: Supermarket



Question:

Parts are made every 10 minutes. 90% are Type A and 10% are type B. The inspection time for A is 6 ± 2 mins and for B is 10 ± 2 mins. Both have 10% rejection rate. Simulate for 100 parts using GPSS.

Question:



Simulation in Python

- **Python** is a general-purpose programming language widely used for simulation and modeling.
- It provides libraries for **discrete-event**, **continuous**, and **agent-based** simulations.
- No special compiler or simulation-specific syntax required, unlike GPSS
- It combines the flexibility of a general-purpose language with powerful simulation libraries.

Simulation in Python

Popular Python simulation libraries include:

- **SimPy** : process-based discrete-event simulation
- **SciPy** : mathematical modeling and numerical simulation
- **NumPy** : efficient numerical computations
- **Matplotlib** : visualization of simulation results

Simulation in Python

SimPy

- SimPy is an open-source, process-based discrete-event simulation (DES) library for Python.
- It allows users to model and simulate real-world systems such as queues, manufacturing processes, healthcare systems, transportation, and communication networks.

Key concepts:

1. **Environment** : Manages simulation time and event scheduling.
2. **Process** : Represents an active entity or sequence of actions.
3. **Event** : Action that trigger after a certain amount of simulated time has passed, or until specific conditions are met..
4. **Resource** : Models shared resources such as servers or machines.
5. **Timeout** : Delays a process for a specified simulation time.

Simulation in Python

Why Python for Simulation?

- 1. Easy to learn and use:** Simple and readable syntax makes simulation models easier to develop and maintain.
- 2. Rich ecosystem of libraries:** Libraries such as SimPy, NumPy, SciPy, Pandas, and Matplotlib support simulation, data analysis, and visualization.
- 3. Supports multiple simulation types:** Can be used for discrete-event, continuous, Monte Carlo, and agent-based simulations.
- 4. Powerful data analysis and visualization:** Easily analyze simulation results and create graphs, charts, and reports.
- 5. Open source and cross-platform:** Free to use and runs on Windows, macOS, and Linux.

Simulation Software

- All the simulation packages described in later subsections run on a PC under Microsoft Windows 2000 or XP.
- Common characteristics:
 - Graphical user interface, animation and automatically collected outputs to measure system performance.
 - Most provide statistical analysis, e.g., confidence intervals for performance measures and comparisons.
- All packages considered here take the process-interaction worldview, a few also allow event-scheduling models and mixed discrete-continuous models.
- For animation, some emphasize scale drawings in 2-D or 3-D; others emphasize iconic-type animation based on process-flow diagram.
- Almost all offer dynamic business graphing, e.g., time lines, bar charts and pie charts.

Simulation Software

- Software packages:

- Arena
- AutoMod
- Delmia/QUEST
- Extend
- Flexsim
- Micro Saint
- ProModel
- Simul8
- WITNESS

Arena

- Arena can be used for simulating discrete and continuous systems.
- At the heart of Arena is the SIMAN (SIMulation ANalysis) simulation language.
- The Arena Basic Edition:
 - For modeling business processes and other systems in support of high level analysis needs.
 - It represents process dynamics in a hierarchical flowchart and stores system information in data spreadsheets.
- The Arena Standard Edition:
 - For modeling more detailed discrete and continuous systems.
 - Models are built from graphical objects called modules to define system logic and physical components.
 - Includes modules focused on specific aspects of manufacturing and material-handling systems.
- The Arena Professional Edition:
 - With capability to craft custom simulation objects that mirror components of real system including terminology, process logic, data, performance metrics and animation.

Arena

- The Arena family includes:
 - OptQuest: an optimization software package.
 - Arena Contact Center and Arena Packaging: designed specifically to model call centers and high-speed production lines.
- Arena's Input Analyzer automates the process of selecting the proper distribution and its inputs.
- The Output Analyzer and Process Analyzer automate comparison of different design alternatives

AutoMod

- AutoMod Product Suite includes:
 - AutoMod simulation package, AutoStat for experimentation and analysis, and AutoView for making AVI moves of the built-in 3-D animation.
- Main focus: manufacturing and material-handling systems (has built in templates.)
- Also contains a full simulation programming language.
- Strength: detailed, large models used for planning, operational decision support, and control-system testing.
- An AutoMod model consists of one or more systems.
 - A system can be either a process system, in which flow and control logic are defined, or a movement system based on one of the material-handling templates.
 - A model may contain any number of systems, which can be saved and reused as objects in other models.
- AutoStat provides a complete environment for the user to define scenarios, conduct experimentation and perform analysis.
- Optimization is based on an evolutionary strategies algorithm.

Extend

- Extend OR, Industry, and Suite are used for simulating discrete and mixed discrete-continuous systems.
- Extend CP is for continuous modeling only.
- Extend combines a block diagram approach to model-building with a development environment for creating new blocks.
 - Models are built by placing and connecting blocks, and entering the parameters on the block's dialog window.
 - For creating new blocks, Extend comes with a compiled C-like programming environment.
- Input parameters can be changed interactively during a model run and can come from external sources.
- Provides iconic process-flow animation of the block diagram.
- Has an open architecture and also supports linking to and using code written in external languages

Flexsim

- Flexsim is a discrete-event, object-oriented simulator developed in C++, using Open GL technology.
- Animation can be shown in tree view, 2D, 3D and virtual reality.
- Flexsim is commonly used to:
 - To build models that behave like the actual physical or conceptual systems they represent.
 - To improve production efficiencies and reduce operating costs through simulation, experimentation, and optimization of dynamic flow systems.
 - Engineers and managers use Flexsim to evaluate plant capacity, balance packaging and manufacturing lines, manage bottlenecks.
- The results of each simulation can be analyzed:
 - Graphically through 3D animation, and
 - Through statistical reports and graphs

Micro Saint

- Micro Saint is a general-purpose, discrete-event, network simulation software package for building models that simulate real-life processes.
- It does not use the terminology or graphic representations of a specific industry.
- Model can be built for any process that can be represented by a flowchart diagram. Data used are defined by the user.
- It provides two views of the simulation model:
 - Network diagram view shows the process flowchart in action.
 - Actionview provides a realistic 2-D picture of the process.
- OptQuest optimization is included with Micro Saint and is designed to automatically search for and find optimal or near-optimal solutions of the model.

ProModel

- A simulation and animation tool designed to model manufacturing systems.
 - Has manufacturing-oriented modeling elements and rule-based decision logic.
 - The modeling elements in ProModel are parts (entities), locations, resources, path networks, routing and processing logic, and arrivals.
- Includes logic for automatically generating cost data associated with a process.
- Also comes with an output viewer.
- Its runtime interface allows a user to define multiple scenarios for experiments.
- It offers 2-D animation with an optional 3-D like perspective view.
- The company also offers MedModel for healthcare systems and ServiceModel for service systems.

Delmia/QUEST

- Delmia/QUEST is a manufacturing-oriented simulation package.
 - Combines an object-based, true 3-D simulation environment with a graphical user interface for material-flow modules.
 - Incorporates 2-D and 3-D CAD geometry to create a virtual factory environment.
- The company also offers a number of workcell simulators:
 - IGRIP® for robotic simulation and programming.
 - PROCESS ENGINEERTM for process-planning.
- Simulation Control Language (SCL): allows expert users to define customer behavior and to gain control over simulation.
- Batch Control Language (BCL): open architecture allows the advanced user to perform batch simulation runs to automatically collect and tabulate data.
- Output is available both numerically and visually.

SIMUL8

- SIMUL8 models are created by drawing the flow of work with the computer mouse, using a series of icons and arrows to represent the resources and queues in the system.
- Main focus is service industries where people are processing transactions.
- The company's goal is to spread simulation very widely across businesses.
- Have very different pricing and support policies.
- Contains features that watch how the product is being used.
- The product is available at two levels: Standard and Professional

WITNESS

- WITNESS has separate versions for manufacturing and service industries.
- WITNESS models are based on template elements.
 - Elements may be customized and combined into module elements and templates for reuse.
 - Displayed in a 2-D layout animation with multiple windows and display layers.

Experimentation and Statistical-Analysis Tools

- Virtually all simulation packages offer support for statistical analysis of simulation output.
- In recent years, many packages have added optimization as one of the analysis tools.
 - Optimization is used to find a “near-optimal” solution.
 - User must define an objective or fitness function, e.g. cost.
 - Recent advances in the field of metaheuristics has offered new approaches to simulation optimization.
- Products discussed:
 - Arena’s Output and Process Analyzer
 - AutoStat
 - OptQuest
 - SimRunner

Arena's Output and Process Analyzer

- Output Analyzer
 - Provides confidence intervals, comparison of multiple systems, and warmup determination to reduce initial condition bias.
- Process Analyzer
 - Adds sophisticated scenario-management capabilities to Arena for comprehensive design of experiments.
 - Allows a user to define scenarios, make the desired runs, and analyze the results.
- OptQuest is used for optimization

OptQuest

- An optimization tool.
- OptQuest is based on a combination of methods: scatter search, tabu search, linear-integer programming, and neural networks.
- The combination of methods allows the search process to escape local optimality in the quest for the best solution.
 - Scatter search: Population_based approach- Creates new solutions with combining existing solutions
 - Tabu search: is then superimposed to prohibit the search from reinvestigating previous solutions
 - Neural Network: Screens out solutions likely to be poor

AutoStat

- AutoStat is the run manager and statistical-analysis product in the AutoMod product family.
- It provides a number of analyses .. Including warm-up determination, absolute and comparison confidence intervals, design of experiments, sensitivity analysis.
- The evolutionary-strategies algorithm used is well suited to find a near-optimal solution without getting trapped at a local optimum.
- An end user can define any number of scenarios by defining factors and their range or values.
- AutoStat supports correlated sampling using common random numbers.
- AutoStat is capable of distributing simulation runs across a local area network and pulling back all results to the user's computer.

SimRunner

- SimRunner was developed by PROMODEL Corporation.
 - Available for ProModel, MedModel and ServiceModel.
- Uses genetic algorithms and evolutionary strategies.
- Manipulates the input factors within boundaries specified by the user seeking to optimize the objective function.
- Also has a utility for helping users estimate:
 - The end of the warm-up phase
 - The number of replications needed to obtain an estimate of the objective function's mean value to within a specified percentage error and confidence level.

References

- Banks, Carson, Nelson & Nicol, “ Discrete-Event System Simulation”